

PID-Controlled DC Motor Speed Control

MATLAB & Simulink Simulation — Control Systems Project

Nakul | B.E. Electrical and Electronics Engineering
IEEE Member | Embedded Systems & Power Electronics

Metric	Value	Metric	Value
Target speed	10 rad/s	Rise time	0.132 s
Settling time	0.257 s	Overshoot	1.03 %
Steady-state error	~0 %	Phase margin	~95°
Gain margin	> 20 dB	Controller type	PID (Kp=100, Ki=200, Kd=10)

1. Introduction and Objectives

This project demonstrates the complete control engineering workflow for a DC motor speed regulation system. Starting from first-principles mathematical modelling, the work progresses through open-loop characterisation, PID controller design, closed-loop performance analysis, frequency-domain stability verification, and disturbance rejection testing. The project targets skills directly relevant to roles in motor drive design, power electronics, embedded control, and automotive/industrial automation systems.

Objectives:

- Derive the DC motor transfer function from physical equations
- Characterise open-loop behaviour and identify control requirements
- Design and tune a PID controller to achieve fast, accurate speed tracking
- Quantify closed-loop performance using standardised metrics
- Verify stability using Bode analysis (gain margin, phase margin)
- Demonstrate disturbance rejection under load torque variations
- Compare P, PI, and PID to isolate the contribution of each control term

2. DC Motor Mathematical Model

The DC motor is modelled from two governing equations — one for the electrical (armature) subsystem and one for the mechanical (rotor) subsystem:

Electrical (armature circuit):

$$V = L \cdot (di/dt) + R \cdot i + K_e \cdot \omega$$

Mechanical (Newton's second law for rotation):

$$J \cdot (d\omega/dt) = K_t \cdot i - B \cdot \omega$$

Applying the Laplace transform to both equations and eliminating the intermediate current variable yields the second-order transfer function from input voltage $V(s)$ to output angular speed $\Omega(s)$:

$$G(s) = K_t / [(J s + B)(L s + R) + K_e \cdot K_t]$$

$$G(s) = 0.01 / (0.005 s^2 + 0.06 s + 0.1001)$$

The system is inherently second-order because it contains two energy storage elements: inductance L (stores electrical energy in the magnetic field) and inertia J (stores mechanical energy in rotation). Each storage element contributes one pole to the transfer function.

Parameter	Symbol	Value	Unit	Physical meaning
Armature resistance	R	1	Ω	Copper winding loss
Armature inductance	L	0.5	H	Magnetic energy storage
Torque constant	K_t	0.01	N·m/A	Current → torque conversion
Back-EMF constant	K_e	0.01	V·s/rad	Speed → back-voltage
Rotor inertia	J	0.01	kg·m ²	Mechanical energy storage
Viscous friction	B	0.1	N·m·s	Speed-dependent drag

3. Open-Loop Analysis

Before designing the controller, the uncontrolled plant behaviour was characterised using a unit step input (Figure 1). Key observations:

Observation	Value	Implication
DC gain (steady-state speed)	0.0999 rad/s/V	Very low — needs voltage amplification
Settling time (open-loop)	~2.5 s	Too slow for practical applications
Overshoot	None (overdamped)	Friction B dominates, $\zeta > 1$
Setpoint tracking	Not possible	No mechanism to follow reference

The DC gain equals $K_t / (BR + K_e K_t) = 0.01 / (0.1 \times 1 + 0.01 \times 0.01) \approx 0.0999$. This analysis confirmed that closed-loop control was necessary to achieve fast, accurate speed tracking.

4. PID Controller Design

A PID controller was selected because it addresses all three fundamental control requirements simultaneously — speed of response, elimination of steady-state error, and suppression of transient overshoot:

Term	Formula	Acts on	Solves	Risk if excessive
Proportional (P)	$K_p \times e(t)$	Current error	Slow/no response	Overshoot, instability
Integral (I)	$K_i \times \int e(t) dt$	Accumulated past error	Steady-state offset	Windup, oscillation
Derivative (D)	$K_d \times de(t)/dt$	Rate of error change	Overshoot from I term	Noise amplification

Tuning procedure:

Step 1: Set $K_i = K_d = 0$. Increase K_p from 0 until rise time is acceptably fast without large overshoot → $K_p = 100$

Step 2: Set $K_d = 0$. Increase K_i until steady-state error reaches zero → $K_i = 200$

Step 3: Increase K_d to suppress the overshoot introduced by integral action → $K_d = 10$

Step 4: Verify using `stepinfo()` — quantify rise time, settling time, overshoot numerically

Final controller gains:

Gain	Value	Role
K_p	100	Fast response to current error
K_i	200	Eliminates steady-state error completely
K_d	10	Damps overshoot from integral accumulation

5. Closed-Loop Performance Results

5.1 Performance Metrics (stepinfo)

Metric	Value	Industry benchmark	Status
Rise time	0.132 s	< 0.5 s	PASS — 3.8× better
Settling time	0.257 s	< 1.0 s	PASS — 3.9× better
Overshoot	1.03 %	< 5–10 %	PASS — excellent
Peak speed	10.10 rad/s	≤ 110% SP	PASS
Steady-state err	~0 %	< 1 %	PASS — integral action

5.2 Controller Comparison

Three configurations were tested on the identical plant to isolate each term's contribution:

Controller	Peak overshoot	Settling time	Steady-state error	Verdict
P only	Low overshoot	> 2 s	~5%	Cannot track — error remains
PI	~35%	~1.8 s	0%	Tracks but oscillates
Full PID	1.03%	0.257 s	0%	Best — fast, accurate, stable

5.3 Disturbance Rejection

A load torque disturbance was injected at $t = 5\text{ s}$. The closed-loop system maintained speed within 2% of setpoint throughout the disturbance and showed complete recovery. This confirms that the integral term actively suppresses load-induced errors in real time — the critical property for variable-load applications such as conveyor systems, CNC axes, and EV drivetrains.

6. Frequency-Domain Stability Analysis

6.1 Bode Plot — Gain and Phase Margins

The open-loop Bode plot of $C_PID \times G(s)$ was generated to verify stability margins:

Stability metric	Value	Minimum acceptable	Interpretation
Phase margin	~95°	> 45°	System can tolerate large parameter variation
Gain margin	> 20 dB	> 6 dB	Gain can increase 10× before instability
Gain crossover	~10 rad/s	—	Bandwidth of the closed-loop system

6.2 Root Locus

The root locus confirms both closed-loop poles are located in the left-half s-plane at approximately -6 on the real axis. Left-half plane location guarantees stability. The distance from the imaginary axis (real part magnitude) determines the speed of transient decay — the poles at -6 correspond to a time constant of ~0.17s, consistent with the measured settling time.

7. Simulation Limitations and Future Work

Linear model assumption: The transfer function assumes linear motor behaviour. Real motors exhibit nonlinear friction (static, Coulomb, Stribeck), magnetic saturation, and mechanical backlash, especially at low speeds and high loads.

Ideal sensor feedback: The simulation uses perfect feedback with zero noise and delay. Real hardware introduces measurement noise, quantisation error from ADC resolution, and communication delay — all of which degrade derivative term performance.

No actuator saturation modelling: Integral windup during input saturation is a critical real-world problem requiring dedicated anti-windup logic (back-calculation or clamping). This was not modelled beyond basic output clamping.

Hardware validation pending: The next step is deploying the PID to Arduino Uno via Simulink external mode with a Hall effect sensor for real speed feedback, to quantify the simulation-to-hardware gap.

8. Conclusion

This project demonstrates a complete control engineering study of a DC motor speed regulation system. The PID controller achieved a rise time of 0.132s, settling time of 0.257s, and overshoot of just 1.03% — significantly exceeding typical industry benchmarks. The controller comparison isolates the physical contribution of each PID term, frequency-domain analysis confirms robust stability with 95° phase margin, and disturbance rejection testing validates performance under variable load. Together, these results form a complete, verifiable characterisation of the control system suitable for portfolio presentation and technical interview discussion.